

Impacto da Política de Substituição de Cache na Eficiência de Canais Laterais: LRU versus Tree-PLRU

Felipe Gabriel de Carvalho¹, Felipe Silva Barros¹, Gabriel Henrique de Moraes¹
Gustavo Henrique dos Santos Dias¹, Thiago Henrique Gomes Feliciano¹

¹Departamento de Ciência da Computação (DCC)
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Campus Coração Eucarístico – Belo Horizonte – MG – Brasil

Abstract. *Cache replacement policies, although designed with performance in mind, can have security implications by exposing exploitable state metadata as side channels. Xiong et al. (2021) showed that the state of LRU and PLRU policies can be used as a timing channel even in cache hit scenarios, but their evaluation focused mainly on the L1 cache, without comprehensive experimental coverage of L2 and LLC. This work proposes to investigate, through simulation in gem5, the impact of the replacement policy, comparing LRU and Tree-PLRU, on the effectiveness of this side channel at higher levels of the memory hierarchy. Metrics for channel capacity and reliability (throughput and bit error rate) as well as memory hierarchy performance metrics (miss rate, latency, and execution time) will be evaluated to identify possible trade-offs between security and performance associated with the choice of replacement policy.*

Resumo. *Políticas de substituição de cache, embora projetadas com foco em desempenho, podem apresentar implicações de segurança ao expor metadados de estado exploráveis como canais laterais. [Xiong et al. 2021] demonstraram que o estado das políticas LRU e PLRU pode ser utilizado como canal de temporização mesmo em cenários de cache hit, mas sua avaliação concentrou-se principalmente no cache L1, sem cobertura experimental abrangente de L2 e LLC. Este trabalho propõe investigar, por meio de simulação no gem5, o impacto da política de substituição, comparando LRU e Tree-PLRU, na eficiência desse canal lateral em níveis superiores da hierarquia de memória. Serão avaliadas métricas de capacidade e confiabilidade do canal (throughput e taxa de erro de bit) e métricas de desempenho da hierarquia de memória (miss rate, latência e tempo de execução), a fim de identificar possíveis trade-offs entre segurança e desempenho associados à escolha da política de substituição.*

1. Introdução

A hierarquia de memória desempenha um papel importante no desempenho dos processadores modernos, permitindo reduzir o tempo necessário para o acesso a dados utilizados com frequência. Dentro dessa hierarquia, as memórias cache utilizam diferentes políticas de substituição para determinar quais linhas devem ser removidas quando novos dados precisam ser armazenados. Embora essas políticas sejam projetadas principalmente com foco em desempenho, seu estado interno também pode apresentar implicações para a segurança dos sistemas.

Ataques de canal lateral exploram características do comportamento do hardware para obter informações sem acessar diretamente os dados da vítima. Nesse contexto, Xiong et al. (2021) demonstraram que o estado das políticas de substituição LRU e PLRU pode ser utilizado como um canal de temporização e comunicação entre processos. Um aspecto relevante do trabalho é que o vazamento pode ocorrer mesmo quando os acessos resultam em cache hits, indicando que não apenas o conteúdo armazenado na cache, mas também seus metadados e mecanismos internos de substituição, podem representar uma superfície de ataque [Xiong et al. 2021].

Diferentes políticas de substituição, entretanto, mantêm e atualizam essas informações de maneiras distintas. A política LRU (Least Recently Used) busca identificar a linha menos recentemente utilizada, enquanto políticas aproximadas, como Tree-PLRU (Tree-based Pseudo-LRU), utilizam estruturas simplificadas para representar a ordem de utilização das linhas. Essas diferenças podem influenciar tanto o comportamento de um canal lateral baseado no estado da cache quanto o desempenho da hierarquia de memória.

Hipotetiza-se que a política Tree-PLRU, por representar de forma aproximada e simplificada a ordem de uso das linhas de cache, produz um canal lateral com throughput mais baixo e/ou taxa de erro de bit mais alta do que a política LRU, às custas de um possível ganho de desempenho (menor miss rate/latência) devido à sua menor complexidade de atualização de estado.

Dessa forma, este trabalho tem como objetivo investigar o impacto da política de substituição na eficiência de canais laterais baseados no estado da cache, comparando as políticas LRU e Tree-PLRU em um ambiente controlado de simulação. Para isso, serão avaliadas métricas relacionadas à capacidade e confiabilidade do canal, como throughput e taxa de erro de bit (BER), além de métricas de desempenho da hierarquia de memória, como cache miss rate, latência média de acesso e tempo de execução. A partir dessa comparação, busca-se analisar se a escolha da política de substituição influencia a eficiência do canal lateral e identificar um possível trade-off entre segurança e desempenho.

2. Discussão dos Trabalhos Correlatos

2.1. Leaking Information Through Cache LRU States in Commercial Processors and Secure Caches

Xiong, Katzenbeisser e Szefer (2021) investigaram o uso do estado das políticas de substituição LRU e PLRU como canal de temporização e de comunicação entre processos. O principal diferencial do trabalho é demonstrar que o vazamento pode ocorrer mesmo quando os acessos resultam em cache hits, explorando as alterações no estado interno da política de substituição. Os autores realizaram experimentos em processadores Intel e AMD e também analisaram mecanismos de proteção, como Partition-Locked e Random Fill [Xiong et al. 2021].

Apesar de sua contribuição, o estudo apresenta limitações importantes. A avaliação experimental concentra-se principalmente no cache L1, enquanto os níveis L2 e LLC são discutidos, mas não avaliados experimentalmente de forma abrangente. Além disso, parte dos mecanismos de defesa analisados foi avaliada apenas de forma qualita-

tiva. Essas limitações motivam a investigação de arquiteturas mais recentes e de outros níveis da hierarquia de cache [Xiong et al. 2021].

2.2. Generic and Automated Drive-by GPU Cache Attacks from the Browser

Giner et al. (2024) investigaram ataques à cache L2 de GPUs realizados a partir de navegadores utilizando WebGPU. O trabalho demonstra que ataques de cache podem ser realizados em um ambiente mais restrito do que o tradicional cenário de código nativo, utilizando técnicas como Prime+Probe e mecanismos desenvolvidos para identificar características da cache e construir conjuntos de evasão. Os autores também demonstraram a possibilidade de utilizar essas técnicas em ataques contra informações processadas pela GPU [Giner et al. 2024].

A principal diferença em relação a Xiong et al. (2021) está tanto no ambiente de execução quanto no mecanismo explorado. Enquanto Xiong et al. concentram-se no estado LRU/PLRU e em alterações observáveis mesmo com cache hits, Giner et al. exploram alterações relacionadas à ocupação e à expulsão de linhas da cache. Dessa forma, o trabalho amplia o cenário de ameaça para GPUs e aplicações executadas por meio de navegadores [Xiong et al. 2021, Giner et al. 2024].

2.3. Random and Safe Cache Architecture to Defeat Cache Timing Attacks

Hu e Lee (2024) apresentam a arquitetura Random and Safe Cache (RaS) como uma abordagem defensiva contra ataques de temporização. A proposta busca reduzir a previsibilidade do comportamento da cache por meio da substituição das políticas determinísticas tradicionais por mecanismos de seleção aleatória, além de utilizar um Safe History Buffer. A arquitetura foi avaliada por meio de simulações no gem5 contra diferentes técnicas de ataque, incluindo Spectre, Prime+Probe, Flush+Reload e Evict-Time [Hu and Lee 2024].

A vantagem do trabalho está em abordar diretamente a questão da proteção da cache, enquanto outros estudos concentram-se na identificação ou exploração das vulnerabilidades. Entretanto, uma limitação relevante é que a arquitetura RaS não foi diretamente avaliada contra o canal específico apresentado por Xiong et al. (2021), baseado na observação do estado LRU/PLRU por meio de cache hits. Assim, permanece a questão de até que ponto mecanismos de randomização são capazes de eliminar esse tipo específico de canal [Hu and Lee 2024, Xiong et al. 2021].

2.4. Analysis of Secure Caches Using a Three-Step Model for Timing-Based Attacks

Deng, Xiong e Szefer (2019) propuseram um modelo de três etapas para analisar sistematicamente vulnerabilidades relacionadas a ataques de temporização em caches. A metodologia considera diferentes formas pelas quais um atacante pode modificar o estado da cache e posteriormente observar alterações nesse estado, permitindo identificar possíveis canais de comunicação ou vazamento de informações [Deng et al. 2019].

A principal contribuição do trabalho está, portanto, na criação de uma metodologia sistemática de análise, servindo como base para a identificação de diferentes classes de vulnerabilidades. Entretanto, grande parte da avaliação é baseada em modelos e análises teóricas, apresentando uma validação experimental mais limitada em hardware comercial. Dessa forma, embora seja útil para identificar possíveis canais, a exploração prática desses canais ainda depende das características específicas de cada arquitetura [Deng et al. 2019].

2.5. Prime+Retouch: When Cache is Locked and Leaked

Lee, Sang e Kim (2024) apresentaram o ataque Prime+Retouch, demonstrando que informações relacionadas aos metadados da política de substituição podem ser exploradas mesmo quando os dados da vítima estão protegidos contra substituição. O ataque foi demonstrado em arquiteturas Intel x86 e Apple M1, mostrando que mecanismos associados à substituição continuam representando uma possível superfície de ataque em arquiteturas modernas [Lee et al. 2024].

Esse trabalho possui relação direta com Xiong et al. (2021), pois ambos demonstram que proteger apenas os dados armazenados na cache pode não ser suficiente. Entretanto, enquanto Xiong et al. exploram diretamente o estado LRU/PLRU como canal de temporização, Lee et al. exploram os metadados relacionados à política de substituição. A utilização de arquiteturas mais recentes também reforça a importância de verificar se vulnerabilidades relacionadas aos mecanismos de substituição permanecem presentes em diferentes gerações de processadores [Xiong et al. 2021, Lee et al. 2024].

2.6. Comparação dos trabalhos e posicionamento da proposta

Os cinco trabalhos apresentam diferentes perspectivas sobre a segurança de caches. Xiong et al. (2021) demonstram experimentalmente a existência de um canal baseado no estado LRU/PLRU; Giner et al. (2024) ampliam o estudo de ataques de cache para GPUs e navegadores; Hu e Lee (2024) apresentam uma arquitetura defensiva baseada em randomização; Deng et al. (2019) propõem uma metodologia sistemática para identificação de canais de temporização; e Lee et al. (2024) demonstram a exploração de metadados associados às políticas de substituição [Xiong et al. 2021, Giner et al. 2024, Hu and Lee 2024, Deng et al. 2019, Lee et al. 2024].

Em termos metodológicos, também existem diferenças relevantes. Os trabalhos utilizam desde experimentos em hardware real, como Xiong et al. e Lee et al., até simulações e modelos, como Hu e Lee e Deng et al. Já Giner et al. exploram um cenário de ataque mais restrito, utilizando um navegador e uma GPU. Em conjunto, essas abordagens mostram que diferentes componentes e níveis da hierarquia de cache podem apresentar comportamentos relevantes para a segurança [Xiong et al. 2021, Giner et al. 2024, Hu and Lee 2024, Deng et al. 2019, Lee et al. 2024].

Nesse contexto, o presente trabalho busca se posicionar principalmente a partir das lacunas identificadas no estudo de Xiong et al. (2021). A proposta é investigar de forma mais ampla o canal relacionado às políticas de substituição, considerando níveis superiores da hierarquia, como L2 e LLC, microarquiteturas mais recentes e diferentes mecanismos de randomização e proteção [Xiong et al. 2021]. Também se pretende investigar o comportamento observado em determinados cenários AMD nos quais o protocolo sem memória compartilhada não apresentou um sinal observável.

Assim, diferentemente dos trabalhos que apresentam uma técnica específica de ataque, uma defesa ou uma metodologia de análise, a proposta busca ampliar e atualizar a avaliação experimental do canal de temporização associado ao estado das políticas de substituição, verificando sua relevância em arquiteturas mais recentes e diante de mecanismos modernos de proteção [Xiong et al. 2021].

3. Metodologia

A metodologia experimental foi definida com o objetivo de avaliar se diferentes políticas de substituição de cache influenciam a capacidade de utilização do estado da cache como canal lateral. Para isso, serão comparadas as políticas LRU (*Least Recently Used*) e Tree-PLRU (*Tree-based Pseudo-LRU*) sob as mesmas condições arquiteturais e de execução.

A avaliação será realizada em duas dimensões. A primeira investiga a capacidade de transmissão e a confiabilidade do canal lateral, considerando métricas como *throughput* e taxa de erro. A segunda avalia o impacto das políticas no desempenho da hierarquia de memória, considerando *cache misses*, latência e tempo de execução.

3.1. Simulador utilizado

O simulador escolhido para este trabalho é o gem5, simulador arquitetural amplamente utilizado em pesquisas de sistemas de memória e hierarquia de cache. Ele foi escolhido por dois motivos principais:

- É o simulador utilizado no artigo-base (*Leaking Information Through Cache LRU States in Commercial Processors and Secure Caches*), o que facilita a comparação direta com os resultados originais;
- Permite configuração detalhada e customizável de políticas de substituição de cache, necessária para comparar LRU e Tree-PLRU sob as mesmas condições arquiteturais.

3.2. Arquitetura simulada

A arquitetura experimental será composta por um processador de núcleo único para os experimentos básicos.

A hierarquia de memória será composta por caches L1 de instruções e dados e por um nível de cache compartilhado, representado inicialmente pela L2. O nível compartilhado será utilizado como principal ambiente para investigação do canal lateral, pois possibilita que diferentes processos tenham acesso às mesmas estruturas de cache e, conseqüentemente, possam interferir no estado de substituição de um mesmo conjunto.

A configuração inicial será ser definida conforme a Tabela 1.

3.3. Variáveis

3.3.1. Variável independente

A principal variável independente será a política de substituição da cache, assumindo dois valores: LRU e Tree- PLRU.

3.3.2. Variáveis dependentes

Métricas relacionadas ao canal lateral:

- Throughput do canal (bits/s transmitidos corretamente);
- Taxa de erro de bit — Bit Error Rate (BER);
- Quantidade de bits corretamente recuperados por execução.

Tabela 1. Configuração arquitetural inicial utilizada nos experimentos.

Componente	Parâmetro	Configuração
Processador	Núcleos	1
	Frequência	2 GHz
	Processos em execução	2 (1 emissor, 1 receptor)
Cache L1 (Instruções)	Capacidade	32 KB
	Associatividade	2-way
	Latência	2 ciclos
Cache L1 (Dados)	Capacidade	32 KB
	Associatividade	2-way
	Latência	2 ciclos
Cache L2 (Compartilhada)	Capacidade	256 KB
	Associatividade	8-way
	Latência	10 ciclos
	Política de substituição	LRU/Tree-PLRU
Memória principal	Tecnologia	DDR4
	Latência	100 ns
Geral	Tamanho da linha de cache	64 bytes

Métricas relacionadas ao desempenho da hierarquia de memória:

- Taxa de cache miss (miss rate);
- Latência média de acesso;
- Overhead de execução em relação ao baseline (LRU);
- Tempo total de execução.

3.4. Workloads

Serão utilizados dois tipos de workload, com propósitos complementares:

Workload controlado (Sender/Receiver): Um par de processos desenvolvido especificamente para este experimento, no qual um processo Sender codifica uma sequência de bits conhecida por meio de acessos deliberados a um conjunto específico da cache compartilhada, e um processo Receiver observa o estado/comportamento desse conjunto para recuperar a sequência transmitida. Esse workload permite demonstrar e quantificar diretamente a existência e a eficiência do canal lateral.

Workload representativo (benchmark): Um benchmark de propósito geral, com uso intensivo de memória, será utilizado para avaliar o impacto de cada política de substituição no comportamento normal da cache (fora do contexto do canal lateral), permitindo mensurar o custo de desempenho de cada política.

3.5. Protocolo de transmissão do canal lateral

A transmissão de informação pelo canal lateral seguirá um protocolo baseado em intervalos de tempo fixos (slots). Em cada slot, o Sender decide, de acordo com o bit a ser transmitido, se acessa (bit 1) ou não (bit 0) um conjunto específico da cache compartilhada. O Receiver, ao final de cada slot, verifica o estado desse conjunto (ou realiza um acesso de sondagem, dependendo da estratégia adotada) para inferir o bit transmitido.

3.6. Estratégia de análise dos resultados

A análise dos resultados será realizada por meio da comparação entre as políticas de substituição LRU e Tree-PLRU, mantendo-se as mesmas condições experimentais para ambas. Dessa forma, busca-se isolar o efeito da política de substituição sobre o comportamento do canal lateral e sobre o desempenho da hierarquia de memória.

Para cada workload, inicialmente será realizada a execução utilizando a política LRU, considerada como baseline. As métricas definidas serão coletadas e armazenadas. Em seguida, o mesmo workload será executado utilizando a política Tree-PLRU, mantendo-se inalterados os demais parâmetros da arquitetura e do experimento. As mesmas métricas serão coletadas para possibilitar uma comparação direta entre as duas políticas.

Cada configuração experimental, definida pela combinação entre política de substituição e workload, será executada múltiplas vezes, buscando reduzir a influência de variações ocasionais e aumentar a confiabilidade dos resultados. A sequência geral da análise será:

1. executar o workload utilizando a política LRU;
2. registrar as métricas obtidas;
3. executar o mesmo workload utilizando a política Tree-PLRU;
4. registrar as mesmas métricas;
5. repetir as execuções para cada configuração experimental;
6. calcular as medidas estatísticas dos resultados;
7. comparar LRU e Tree-PLRU quanto às métricas de canal lateral e de desempenho;
8. avaliar o possível *trade-off* entre resistência ao canal lateral e desempenho da cache.

Para cada métrica, serão calculadas, quando aplicável, a média e o desvio padrão. Essas medidas permitirão não apenas comparar os valores médios obtidos, mas também avaliar a variabilidade dos resultados entre diferentes execuções.

A comparação será realizada considerando dois grupos principais de métricas. Para o canal lateral, serão analisadas a capacidade de transmissão, o *throughput*, a taxa de erro de bit (BER), a quantidade de bits corretamente recuperados e a confiabilidade da recuperação da informação. Para o desempenho da hierarquia de memória, serão analisados o *miss rate*, a latência média de acesso, o tempo total de execução e o overhead de desempenho em relação ao baseline LRU.

Dessa forma, será possível avaliar o possível *trade-off* entre segurança e desempenho, verificando se uma eventual redução na eficiência do canal lateral é acompanhada por um custo arquitetural significativo.

Referências

- Deng, S., Xiong, W., and Szefer, J. (2019). Analysis of secure caches using a three-step model for timing-based attacks. *Journal of Hardware and Systems Security*, 3:397–425.
- Giner, L., Czerny, R., Gruber, C., Rauscher, F., Kogler, A., Braga, D. D. A., and Gruss, D. (2024). Generic and automated drive-by gpu cache attacks from the browser. In

Proceedings of the 19th ACM Asia Conference on Computer and Communications Security, pages 128–140. ACM.

Hu, G. and Lee, R. B. (2024). Random and safe cache architecture to defeat cache timing attacks.

Lee, J., Sang, F., and Kim, T. (2024). Prime+retouch: When cache is locked and leaked. In *arXiv preprint arXiv:2402.15425*.

Xiong, W., Katzenbeisser, S., and Szefer, J. (2021). Leaking information through cache hit states in commercial processors and secure caches. *IEEE Transactions on Computers*.